ST. CLARET COLLEGE (AUTONOMOUS), BENGALURU
Department of Bachelor of Computer Applications (BCA)

Vacation Mini Project Report

ElectVote

A Real-Time Browser-Based Online Voting System

Democracy in Your Hands.

Submitted By	**NITHISH**
Roll Number	**24BCA097**
Department	**Bachelor of Computer Applications (BCA)**
Semester	**4th Semester**
Submission	**July 2026**

https://online-voting-systemsz.netlify.app/

CERTIFICATE

This is to certify that the mini project entitled

ELECTVOTE — ONLINE VOTING SYSTEM

has been successfully developed and submitted by **NITHISH** (Roll No. **24BCA097**), a student of the 4th Semester, Bachelor of Computer Applications (BCA), St. Claret College (Autonomous), Bengaluru.

This mini project was developed independently as part of self-learning during the vacation period. The work is original and has not been submitted elsewhere for any academic examination or award.

St. Claret College (Autonomous), Bengaluru

DECLARATION

I, **NITHISH**, Roll No. **24BCA097**, 4th Semester, BCA, St. Claret College (Autonomous), Bengaluru, declare that the mini project **"ELECTVOTE — ONLINE VOTING SYSTEM"** is a record of original work done by me during the vacation period.

I declare that:

- This project was carried out independently, without any formal institutional guidance.
- The work is genuine and has not been copied from any source without acknowledgement.
- This project has not been submitted for any degree or academic title elsewhere.
- All references have been duly cited and acknowledged.
- The conclusions drawn are based entirely on my own understanding and research.

Place: Bengaluru

Date: July 2026

Signature of the Student

NITHISH

Roll No: 24BCA097 | 4th Semester, BCA

ACKNOWLEDGEMENT

I thank all those who encouraged and supported me throughout the development of this project.

I am grateful to the open-source developer community and the educational resources available through MDN Web Docs, Stack Overflow, and YouTube, which guided my understanding of real-time vote tracking, Chart.js integration, localStorage-based state management, and secure admin authentication patterns in JavaScript.

I thank my family for their constant support and encouragement throughout the vacation period.

Building a complete election system — with live result tracking, candidate management, an admin panel, and data visualisation — entirely in the browser was a deeply rewarding and instructive experience that significantly broadened my practical understanding of front-end application development.

NITHISH

Roll No: 24BCA097 | 4th Semester, BCA

St. Claret College (Autonomous), Bengaluru

ABSTRACT

Democratic participation depends on accessible, transparent, and trustworthy voting mechanisms. Traditional paper-based voting is slow, geographically restricted, and difficult to audit in real time. Existing digital voting platforms are typically centralised, server-dependent systems requiring installation, registration, and backend infrastructure. ElectVote proposes an alternative: a fully browser-based, client-side election simulation platform that demonstrates the core mechanics of a democratic voting system with real-time result tracking and administrative controls.

ElectVote is a dark-themed, four-module web application comprising a Hero Landing Page, a Voting Module, a Live Results Dashboard, and an Administration Panel. It allows users to browse candidates from multiple parties, cast YES/NO votes for each candidate, view live election results with animated charts, and manage the candidate pool through an admin interface — all without any server or database.

Key Features:

- Home page with live election status, leading candidate widget, and real-time vote counter.
- Voting module: browse all candidates with search and party filter; cast YES/NO votes per candidate.
- One-vote-per-candidate enforcement: each voter can cast exactly one vote (YES or NO) per candidate.
- Live Results Dashboard: total votes, YES/NO breakdown, current leader card, and two Chart.js charts.
- Yes vs No doughnut chart and Votes per Candidate grouped bar chart updating in real time.
- Administration Panel: add new candidates with photo, party logo, manifesto, age, and party colour.
- Admin authentication with a password-protected login gate.
- Dark/Light mode toggle saved to localStorage.
- Fully client-side — no server, no database, no login required for voting.

Technologies Used:

HTML5, CSS3, Vanilla JavaScript, Chart.js, localStorage API. Deployed on Netlify.

TABLE OF CONTENTS

Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
Table of Contents	6
Chapter 1 – Introduction	7
Chapter 2 – Objectives of the Project	8
Chapter 3 – Existing System	9
Chapter 4 – Proposed System	10
Chapter 5 – Modules and Features	11
Chapter 6 – Technologies Used	15
Chapter 7 – Conclusion	17

INTRODUCTION

1.1 Background

Free and fair elections are the cornerstone of democratic governance. As digital technology permeates every aspect of civic life, the demand for accessible, transparent digital voting tools has grown significantly. While fully secure national e-voting systems require sophisticated cryptographic infrastructure, simulation platforms that demonstrate the mechanics of election systems serve an important educational and prototyping purpose.

ElectVote was developed as a practical demonstration of how a browser-based election system — encompassing candidate management, real-time voting, live result analytics, and administrative oversight — can be built using only front-end web technologies.

1.2 About the Project

ElectVote is a fully client-side online voting simulation application built for the 2026 General Election scenario. It supports seven candidates across five parties, real-time vote tallying, live chart updates, and an admin panel for candidate management. The dark-themed interface with orange accents is both visually striking and functionally comprehensive.

Live at: **https://online-voting-systemsz.netlify.app/** — deployed on Netlify.

1.3 Scope

- Real-time YES/NO voting per candidate with one-vote-per-candidate enforcement.
- Live election results with Chart.js doughnut and bar chart visualisations.
- Current leader identification and dynamic vote counters.
- Admin panel for adding, editing, and removing candidates with full metadata.
- Dark/light mode, party filter, and candidate search functionality.
- All data persisted locally in the browser via localStorage.

OBJECTIVES OF THE PROJECT

The following objectives guided the design and development of ElectVote:

1. To simulate a realistic digital election environment: Recreate the core mechanics of a democratic election — candidate listing, vote casting, result tallying, and leader tracking — within an accessible browser-based platform.

2. To implement fair voting constraints: Enforce a strict one-vote-per-candidate policy so that each user can cast exactly one vote (YES or NO) per candidate, preventing vote manipulation or double voting.

3. To provide real-time election analytics: Display live vote counts, YES/NO ratios, the current leading candidate, and graphical breakdowns that update dynamically as votes are cast.

4. To build an administrative management interface: Provide a password-protected admin panel through which election administrators can add new candidates with full metadata — name, age, party, manifesto, photo, and party logo.

5. To deliver intuitive candidate browsing: Allow voters to search candidates by name and filter by party affiliation, making it easy to locate specific candidates in a multi-party election.

6. To design a visually engaging civic interface: Create a dark-themed, professional UI that communicates the seriousness of an election platform while remaining accessible and easy to navigate.

7. To demonstrate complete front-end application development: Apply HTML5, CSS3, JavaScript, and Chart.js to build a multi-module, stateful application with real-time data flow, all without any server-side component.

EXISTING SYSTEM

3.1 Current Voting Methods

Election systems currently in use span a wide spectrum from traditional paper ballots to centralised government e-voting platforms, each carrying notable limitations:

a) Paper-Based Voting

Traditional ballot-based elections require physical presence at polling booths, are time-consuming to count, and provide no real-time result visibility. Geographic and mobility constraints reduce participation rates among certain demographics.

b) Centralised Government E-Voting Platforms

National e-voting systems such as those used in Estonia or pilots in India require substantial server infrastructure, cryptographic identity verification, and significant investment in security auditing. They are not replicable as lightweight educational tools.

c) Generic Survey or Form Tools

Tools like Google Forms or SurveyMonkey can be used for simple polls but lack election-specific features such as party-based candidate organisation, live result analytics, YES/NO voting mechanics, and administrative candidate management.

d) No Accessible Simulation Tool

No free, browser-based, installation-free tool exists that simulates a complete multi-candidate, multi-party election with real-time analytics and administrative controls for educational or demonstration purposes.

3.2 Limitations Summary

- Paper voting is slow, geographically restricted, and produces no real-time results.
- Government e-voting requires extensive infrastructure and is not accessible for simulation.
- Generic form tools lack election-specific features and analytics.
- No lightweight, educational, browser-based voting simulation tool is freely available.
- Existing tools do not combine voting, live analytics, and admin management in one platform.

PROPOSED SYSTEM

4.1 Overview

ElectVote proposes a fully browser-based, client-side election simulation that combines candidate browsing, real-time YES/NO voting, live result analytics, and administrative candidate management in a single, instantly accessible web application. It requires no server, no database, no installation, and no login for standard voting participation.

4.2 Advantages

- **Real-Time Results:** Vote tallies, chart visualisations, and the current leader update instantly after every vote.
- **Fair Voting Enforcement:** One-vote-per-candidate constraints are enforced in localStorage, preventing duplicate voting within a session.
- **Multi-Party Structure:** Seven candidates across five parties are organised with searchable and filterable listings.
- **Admin Control:** A password-protected admin panel enables full candidate lifecycle management without a backend.
- **Zero Setup:** Open the URL and vote immediately — no account, no installation, no server required.
- **Educational Value:** Demonstrates the complete mechanics of a digital election system for academic study.

4.3 Architecture

ElectVote is a single-page application with four distinct views navigated via a top navigation bar: Home, Vote, Results, and Admin. All election state — candidate data, vote counts, admin session, and theme preference — is persisted in the browser's localStorage. Chart.js renders live analytics from this state. The admin panel is protected by a JavaScript-based password check.

MODULES AND FEATURES

ElectVote comprises four primary modules, each accessible via the top navigation bar.

5.1 Home Module

The landing page establishes the election context with a bold typographic hero section reading "Democracy in Your Hands." A live badge — '2026 GENERAL ELECTION · LIVE' — signals the active election status. Three floating widgets display real-time data: a YES Votes counter with a progress bar, a Leading Candidate card showing the current frontrunner with their vote count, and an Election Status indicator confirming votes are recorded in real time.

Three statistics counters at the bottom of the hero section display Total Votes Cast, total Candidates, and total Parties — all updating live as votes are submitted.



Figure 5.1 – Home Module with Hero Section and Live Election Widgets

5.2 Vote Module — All Candidates

The Vote module displays all election candidates in a card grid. Each candidate card shows the candidate's photo, name, age, party name (with party colour coding), manifesto extract, YES and NO vote counts, a vote progress bar, and two action buttons.

Voting Controls:

- **YES:** Cast a positive vote for the candidate.
- **NO:** Cast a negative vote for the candidate.
- Once a vote is cast, the buttons are replaced with a ‘You voted YES/NO’ confirmation badge and the vote cannot be changed.

Search and Filter:

A search bar allows filtering candidates by name in real time. A party dropdown filters the display to show only candidates from a selected party (National Progressive Party, People’s Democratic Front, Bharatiya Jan Seva, Greens & Sustainability Party, or All Parties).

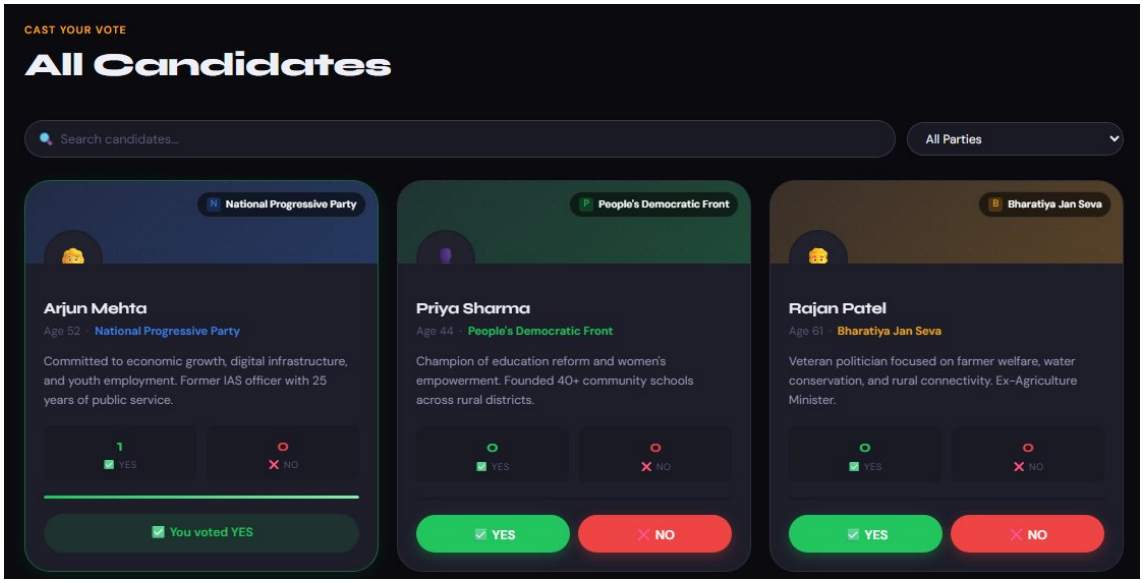


Figure 5.2 – Vote Module Showing Candidate Cards with YES/NO Controls

5.3 Live Results Module

The Results module provides a comprehensive real-time election analytics dashboard.

Summary Metric Cards:

- **Total Votes:** Cumulative votes across all candidates and both vote types.
- **Total YES:** Sum of all YES votes across the election.
- **Total NO:** Sum of all NO votes across the election.
- **Candidates:** Total number of candidates currently in the system.

Current Leader Card:

A highlighted card identifies the candidate with the most YES votes, displaying their name, party, age, and vote breakdown with Current Leader, YES count, and NO count badges.

Charts:

- **Yes vs No — Overall:** A Chart.js doughnut chart showing the proportion of YES to NO votes across all candidates.
- **Votes per Candidate:** A Chart.js grouped bar chart displaying YES (green) and NO (red) vote counts for each candidate individually.

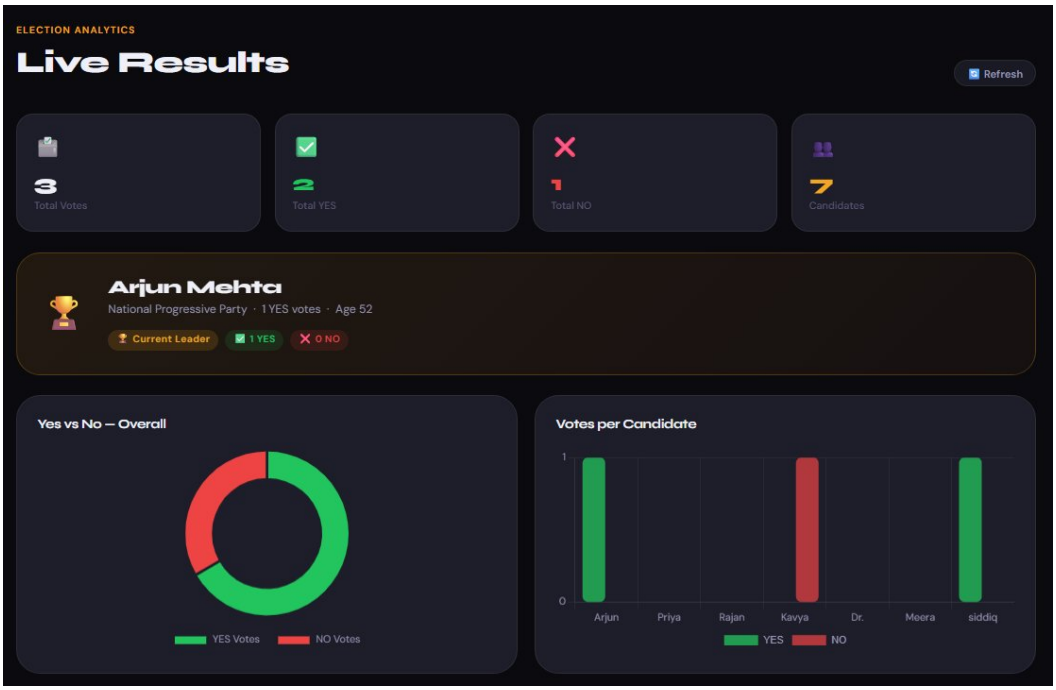


Figure 5.3 – Live Results Dashboard with Metric Cards and Chart.js Visualisations

5.4 Administration Panel

The Admin module is accessible via the Admin button in the navigation bar and is protected by a password gate. Upon successful authentication, the administrator gains access to the full candidate management interface.

Add New Candidate Form:

- Photo upload (JPG/PNG, max 2MB) for the candidate's profile image.
- Party logo upload (PNG recommended) for the party symbol.
- Candidate name, age, party name, and party colour picker.
- Manifesto / description text area for the candidate's platform.

Current Candidates List:

All existing candidates are listed with their name, party, and age. A delete button (red trash icon) removes a candidate from the system. All changes are immediately persisted to localStorage.

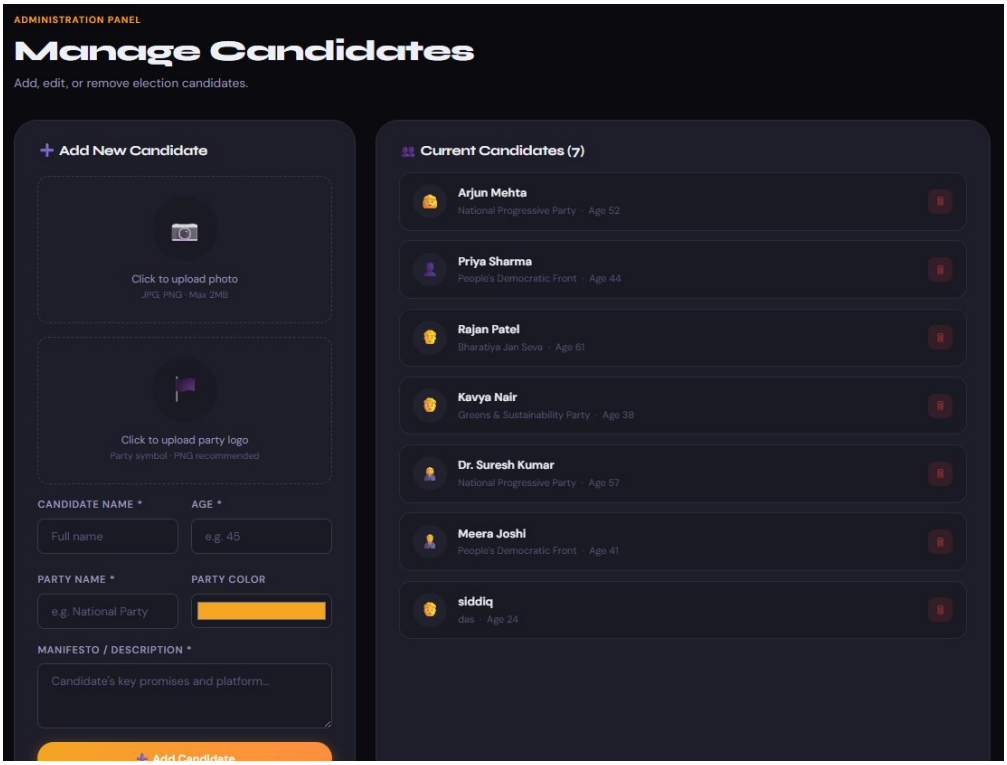


Figure 5.4 – Administration Panel with Add Candidate Form and Candidate List

TECHNOLOGIES USED

ElectVote was developed exclusively using native front-end web technologies. No back-end server, database, or external API is used.

6.1 HTML5

HTML5 provides the structural foundation of all four application modules. Semantic elements organise the navigation bar, candidate card grid, results dashboard, and admin form. The File API enables candidate photo and party logo upload from the local file system. Data attributes on candidate cards store vote state for JavaScript access.

6.2 CSS3

CSS3 delivers ElectVote's striking dark navy-and-orange aesthetic. CSS Custom Properties manage both the dark and light theme colour systems, switching instantly via a class toggle on the root element. CSS Grid and Flexbox handle the responsive candidate card grid and results layout. CSS transitions animate the vote confirmation badge appearance and card hover effects. Each party's colour is applied dynamically via inline CSS Custom Properties derived from the admin panel settings.

6.3 JavaScript (ES6+, Vanilla)

All application logic is implemented in Vanilla JavaScript. Key implementations include: dynamic candidate card rendering from a JSON data array stored in localStorage, vote casting with immediate localStorage update and UI re-render, one-vote-per-candidate constraint enforcement using a voted-candidates map in localStorage, real-time leader calculation by scanning all candidate vote counts, live statistics counters updating on every vote event, admin authentication via a password comparison gate, and File API-based image reading for candidate photo preview.

6.4 Chart.js

Chart.js renders the two live analytics charts in the Results module: a doughnut chart showing the YES vs NO split across all votes, and a grouped bar chart displaying the YES and NO counts per candidate. Both charts are instantiated with the current vote data from localStorage and are destroyed and re-created on every page refresh or result update to reflect the latest state.

6.5 localStorage API

The Web Storage API's localStorage interface persists all election state: the candidate roster (including photos as base64 strings), vote counts per candidate, the voted-candidates record for each session, the admin authentication state, and the current theme preference. This enables the election state to survive

page refreshes without any server-side storage.

6.6 Netlify

The application is deployed as a static site on Netlify, providing HTTPS, a custom subdomain, and global CDN distribution with no server-side infrastructure required.

#	Technology	Type	Purpose
1	HTML5	Markup	Structure, File API, candidate card templates
2	CSS3	Styling	Dark/light themes, grid layout, animations
3	JavaScript ES6+	Programming	Voting logic, admin auth, live state management
4	Chart.js	Library	Doughnut and bar chart election analytics
5	localStorage	Browser API	Election state, votes, candidate data persistence
6	Netlify	Platform	Static hosting and CDN deployment

CONCLUSION

ElectVote is a well-conceived and technically accomplished mini project that demonstrates how a meaningful, multi-module civic application can be built entirely on the front-end web platform. By combining real-time vote tracking, live Chart.js analytics, multi-party candidate management, and an authenticated administration panel — all within a single, server-free, browser-based application — the project delivers a complete election simulation that goes well beyond the typical scope of a student vacation exercise.

The one-vote-per-candidate enforcement mechanism demonstrates an understanding of session-based state management and data integrity constraints in a client-side environment. The admin panel's password authentication and candidate lifecycle management show an awareness of role-based access control concepts, even within a purely front-end architecture.

The Chart.js integration — with a live doughnut chart for the YES/NO ratio and a grouped bar chart for per-candidate breakdowns — transforms raw vote data into immediately interpretable visual insights, replicating the kind of election analytics seen in professional media reporting tools.

Completing this project independently during the vacation period has been a formative experience in full-cycle front-end application development. Designing a multi-module architecture, managing complex application state across views, implementing role-based access, and deploying a polished public-facing product has substantially advanced both the technical skills and the product thinking required for professional software development.

In conclusion, ElectVote is a polished, feature-complete, and publicly deployed online voting simulation that fully achieves its stated objectives and stands as a strong demonstration of front-end development competence, civic-minded design thinking, and independent learning initiative.

Live Project: https://online-voting-systemsz.netlify.app/